

Pavan Kumar Gannaju

+91 6300422984 | pavan9542644804@gmail.com | [LinkedIn](#) | [Portfolio](#) | [Github](#)

PROFESSIONAL SUMMARY

Backend and systems engineer (Java, Go, Rust, Python) who checks correctness with fault injection, property-based tests, fuzzing and CI. Built a double-entry ledger with benchmarked concurrency strategies, a Raft-replicated key-value store validated with a linearizability checker, and a capability-based OS kernel whose revocation is asserted in CI. Every figure below comes from a committed, re-runnable measurement.

EXPERIENCE

AI Intern → Research Team Lead — Prodigal AI Technologies Pvt. Ltd., Delhi

Mar 2025 – Nov 2025 | Remote, part-time

- Promoted from AI Intern to Research Team Lead; coordinated a small engineering team through architecture reviews and sprint delivery for internal AI workflow systems.
- Built modular Python backend workflows and API integration utilities for data processing and cross-service coordination; owned request validation and issue resolution end to end.

Gen AI / LLM Intern — Digital Nexus AI, Jersey City, NJ, USA

May 2025 – Sep 2025 | Remote, part-time

- Developed 5+ backend REST APIs in Python and FastAPI supporting inter-service communication and automated workflow execution.
- Supported deployment workflows in Linux cloud environments; contributed to integration testing and defect triage ahead of release cycles.

PROJECTS

LedgerLine — Double-Entry Ledger | Java 21, Spring Boot, PostgreSQL, Kafka, Testcontainers, k6

- Designed a ledger where PostgreSQL enforces zero-sum postings and non-negative balances, with idempotent transfers (request-hash conflict detection) and a transactional outbox to Kafka feeding a deduplicating projection service.
- Benchmarked four concurrency strategies with k6 (raw output committed): on 3 hot accounts, ordered pessimistic locking had 0% failures at p95 171 ms, while optimistic and serializable locking degraded to p95 1.4 s and 2.3 s with about 13% failures; the deliberately unprotected variant lost updates and broke conservation of money.
- Tested failure paths against a real Kafka container and with property-based tests (jqwik) driving the real service; CI runs the full suite.

QuorumKV — Replicated Key-Value Store | Go, gRPC, hashicorp/raft, Porcupine

- Wrote a CRC-framed, fsync-before-ack write-ahead log and proved it with 100/100 randomized process-kill trials: no acknowledged write lost, none invented.
- Built a fault-injection harness (TCP proxy per directed link) over a 3-node Raft cluster and checked recorded client histories for 5 scenarios (leader and minority partitions, rolling restarts, network delay, double-leader attempt) with Porcupine; validated the checker against known-bad histories (12 tests) and modelled errored writes as "may have happened".

Gatekeeper — Fail-Closed LLM Guardrail Gateway (Solo) | Python, FastAPI, PyTorch, Redis, Docker

- Built a prompt-injection and PII gateway (symbolic rules, FAISS search, learned fusion of 8 detectors, server-side privilege resolution, audit log); 1,644 tests, CI with dependency audit and Trivy/SBOM image scanning.
- Profiled it with py-spy, found the ensemble oversubscribing CPU under concurrency, and caught a cache-warmth flaw in my own first benchmark; pinning torch threads raised throughput from about 10.3 to 16.0 rps and cut p95 from about 2.1 s to 1.3 s at 16 concurrent requests (same workload, one machine; p99 unchanged).
- Found and fixed an authorization bypass: a client-supplied role field could set INTERNAL and disable every guardrail; capability is now resolved server-side and the field is rejected with a 422.

Agentic-OS — Capability-Based x86_64 UEFI Kernel (Solo) | Rust (no_std), UEFI, QEMU, GitHub Actions

- Wrote a UEFI-booting kernel where all access goes through per-process capabilities: rights attenuation enforced on derive, O(1) generation-counter revocation, drivers in user space.
- Linux CI builds the kernel, runs host tests, boots it in QEMU and asserts capability revocation, and fuzzes the ext2 parsers; each subsystem is marked Verified in CI, Demonstrated, Partial or Experimental. Runs under QEMU only.
- Traced a non-deterministic CI-only kernel page fault with added instrumentation; fixed a missing LAPIC end-of-interrupt in the timer handler and removed AP-trampoline setup that ran with no application processors.

Enterprise RAG — FP&A Backend Platform | Python, FastAPI, Ollama, Docker

- Replaced serial per-chunk embedding with bounded concurrency, cutting a 181-chunk ingest from 378.7 s to 48.1 s (7.83×); a 1,408-chunk corpus ingests in 87 s. That scale exposed a real defect: a mid-stream connection reset raised OSError, not URLError, and was never retried; fixed with a regression test reproducing it.
- Load-tested real concurrent HTTP traffic to separate rate-limiter policy (29/40 served, 11 rejected with 429) from capacity (40/40 at a raised limit, ~1.5 req/s), showing the ceiling is model generation, not the API layer. Added X-Request-ID tracing and a /ready probe; 343 tests, 98% branch coverage, CI on Python 3.11 and 3.12.

SKILLS

Languages: Java, Go, Rust, Python, C/C++, SQL, JavaScript

Backend & Infrastructure: Spring Boot, FastAPI, gRPC, Kafka, asyncio, REST APIs, Docker, GitHub Actions, AWS, Linux

Databases: PostgreSQL, MySQL, SQLite · **Testing & Verification:** Testcontainers, property-based testing, fuzzing, Porcupine, k6

Core CS: Data Structures & Algorithms, OOP, DBMS, Operating Systems, Computer Networks, Distributed Systems, System Design

EDUCATION

B.Tech, Computer Science & Engineering (AI/ML) — Visvesvaraya College of Engineering and Technology (JNTUH) | April 2026 | CGPA: 8.01/10.0

IIT Ropar – Major in Artificial Intelligence | online programme via Masai/NSDC | 2024 – 2026

CODING & LEADERSHIP

[LeetCode](#) 530+ solved including 86 Hard · [Codeforces](#) 249 problems solved (218 in 2026) · Led a 5-member development team on backend implementation and code reviews · Organised and mentored two university hackathons (50+ participants), providing debugging and algorithmic problem-solving support